

Factora

Funktions- & API-Referenz

E-Invoicing-Plattform · XRechnung · ZUGFeRD · Peppol · DATEV · GoBD
Vollständige technische Referenz

Inhalt

- 01 Überblick
- 02 Rechnungstypen
- 03 Rechnungsformate
- 04 Steuerlogik
- 05 Compliance & Archivierung
- 06 API-Überblick
- 07 Endpoints
- 08 Atomic Invoice — Feldschema
- 09 Webhooks
- 10 Exporte

Factora — Funktions- & API-Referenz

Vollständige technische Referenz der Factora E-Invoicing-Plattform: was die Plattform kann, welche Rechnungsformate sie erzeugt, welche Steuerlogik greift und wie die öffentliche REST-API im Detail funktioniert — inklusive aller Felder, Rechnungstypen und Endpoints.

Diese Referenz ist format- und integrationsneutral. Sie beschreibt die Plattform allgemein, nicht einen einzelnen Anwendungsfall.

Überblick

Factora ist eine E-Invoicing-Plattform für Deutschland und die EU. Sie nimmt Rechnungsdaten in einem strukturierten Format entgegen und erzeugt daraus gesetzeskonforme elektronische Rechnungen nach **EN 16931** — als **XRechnung** (CII), **ZUGFeRD** (PDF/A-3 mit eingebetteter XML) und **Peppol/UBL**. Jede finalisierte Rechnung wird gegen das amtliche Regelwerk geprüft, revisionssicher (GoBD) archiviert und kann per DATEV exportiert werden.

Es gibt zwei Wege, Factora zu nutzen:

- **Web-Anwendung** — Rechnungen im Browser erfassen, finalisieren, versenden.
- **Öffentliche REST-API** — Rechnungserstellung direkt aus einem Fremdsystem (ERP, Warenwirtschaft, Abrechnungssoftware). Ein einziger Aufruf erzeugt eine fertige, geprüfte Rechnung als PDF und XML.

Inhaltsverzeichnis

Plattform & Engine

- Rechnungstypen — alle unterstützten Rechnungsarten und ihre TypeCodes (Rechnung, Korrektur, Abschlag, kumulierte Abschläge, Schlussrechnung, Gutschrift/Self-Billing).
- Rechnungsformate — XRechnung (CII), ZUGFeRD (PDF/A-3), Peppol/UBL; Profile, Prüfung und amtliche Abnahme (KoSIT).
- Peppol BIS 3.0 & EAS-Codes — Customization-/Profile-Kennungen, Gutschriftverfahren (389), EAS-Codes der Teilnehmer-Endpunkte.
- Validierung — XSD und Schematron, geprüfte Regelsätze, Aufbau eines Befunds.
- Steuerlogik — alle USt-Kategorien, Steuersätze, Befreiungsgründe, §13b Reverse Charge, innergemeinschaftliche Lieferung, Ausfuhr, §19 Kleinunternehmer, grenzüberschreitende Logik, Prüfregeln.
- Compliance & Archivierung — GoBD-Unveränderlichkeit, verschlüsseltes Archiv, Änderungsprotokoll, EN-16931-Validierung.

Öffentliche API

- Schnellstart — vom leeren Account zum ersten authentifizierten Aufruf.
- API-Überblick — Basis-URL, Authentifizierung, API-Schlüssel, Sandbox/Live, Idempotenz, Limits, Antwort-Hülle, Fehlercodes, Scopes.
- Sandbox & Testing — kostenlose Trockenläufe, zugelassene Endpunkte, Verhalten des Dry-Runs.
- Endpoints — vollständige Liste aller Endpunkte mit Methode, Zweck, erforderlichem Scope und Verhalten.
- Atomic Invoice — vollständiges Feldschema — jedes Feld des Ein-Aufruf-Rechnungsendpunkts (Kopf, Verkäufer, Käufer, Positionen, Summen, Anhänge) mit Typ, Pflicht/Optional, Default und EN-16931-Bezug.
- Webhooks — Ereignistypen, Signaturprüfung, Zustellung & Wiederholungslogik.
- Exporte — DATEV-Export (Buchungsdaten).

Konventionen in dieser Referenz

- **BT-/BG-Codes** verweisen auf die Geschäftsbegriffe (Business Terms / Groups) der Norm **EN 16931**.
- **TypeCodes** (z. B. 380, 384) stammen aus **UNTDID 1001**, **Steuerkategorien** aus **UNTDID 5305**, **Maßeinheiten** aus **UN/ECE Rec. 20 / UNCL 5272**.
- Geldbeträge werden als Dezimal-Strings mit 2 Nachkommastellen übertragen (z. B. "**1190.00**"), Mengen mit 3 (z. B. "**10.500**"), Sätze mit 2 (z. B. "**19.00**"). Datumsangaben sind **ISO 8601** (**YYYY-MM-DD**).

Interaktive Doku

Die maschinenlesbare OpenAPI-3.1-Spezifikation mit Ausprobier-Funktion ist öffentlich erreichbar unter console.factora.software/api/docs/.

Rechnungstypen

Factora unterstützt die in Deutschland und der EU gebräuchlichen Rechnungsarten. Der Rechnungstyp wird über den **TypeCode (BT-3, UNTDID 1001)** gesteuert und bestimmt die rechtliche Einordnung sowie die Struktur des erzeugten Dokuments.

Unterstützte TypeCodes

TypeCode	Rechnungsart	Bedeutung
380	Rechnung	Standard-Rechnung (Voreinstellung)
384	Korrekturrechnung	Berichtigung einer bereits gestellten Rechnung
386	Abschlagsrechnung	Voraus-/Teilzahlung (auch kumulierte Abschläge)
389	Gutschrift (Gutschriftverfahren / Self-Billing)	Der Leistungsempfänger stellt die Rechnung im Namen des Leistenden aus (§14 Abs. 2 UStG)

Über die öffentliche API wird der Typ im Feld `invoice_header.invoice_type` gesetzt — entweder als TypeCode ("**380**") oder als semantisches Kürzel. Ohne Angabe gilt **380** (Standard-Rechnung).

Die Rechnungsarten im Detail

Standard-Rechnung (380)

Die normale Ausgangsrechnung. Enthält Verkäufer, Käufer, Positionen, Steueraufstellung und Zahlungsangaben. Erzeugt XRechnung-XML und ZUGFeRD-PDF.

Abschlagsrechnung (386)

Für Voraus- oder Teilzahlungen vor vollständiger Leistungserbringung — typisch bei längeren Projekten.

- **Einzelne Abschläge** werden als Abschlagsrechnung mit TypeCode 386 ausgewiesen.
- **Kumulierte Abschläge** werden unterstützt: bereits gestellte Abschläge werden fortlaufend aufsummiert geführt, sodass die Folgerechnung den korrekten kumulierten Stand trägt. Das deckt das im Bauwesen übliche Vorgehen nach **VOB/B §16** ab.

Schlussrechnung

Schließt ein Projekt ab und verrechnet die zuvor geleisteten Abschläge gegen den Gesamtbetrag. Die Schlussrechnung wird als reguläre Rechnung (TypeCode 380) geführt; der Bezug zu den Abschlägen wird über die kumulierte Abschlagsführung hergestellt.

Korrekturrechnung (384)

Berichtigt eine bereits gestellte Rechnung. Über das Feld `invoice_header.parent_invoice` wird die ursprüngliche Rechnung referenziert (BG-3 / BT-25 / BT-26). Im UBL/Peppol-Format wird eine Korrektur als **CreditNote** abgebildet.

Gutschrift / Self-Billing (389)

Das **Gutschriftverfahren** nach §14 Abs. 2 UStG: nicht der Leistende, sondern der **Leistungsempfänger** stellt die Rechnung aus (und vergibt damit auch die Rechnungsnummer). Anwendungsfall z. B. Abrechnung durch den Auftraggeber.

Hinweis zum Reifegrad: Im XRechnung/CII-Pfad ist der TypeCode 389 erzeugbar und amtlich (KoSIT) geprüft. Im internationalen Peppol/UBL-Pfad wird das eigenständige **Peppol-Self-Billing-Profil** (BIS Self-Billing 3.0) erzeugt und gegen das offizielle OpenPeppol-Schematron validiert; die Prüfszenarien dafür pflegt Factora selbst, da weder KoSIT noch OpenPeppol eine fertige Validator-Konfiguration für Self-Billing ausliefert (siehe Rechnungsformate).

Abgrenzung: Begriff „Gutschrift“

In der Praxis wird „Gutschrift“ doppeldeutig verwendet:

- **Kaufmännische Gutschrift / Stornierung** einer Rechnung → das ist eine **Korrekturrechnung (384)**.
- **Gutschrift im umsatzsteuerlichen Sinn (Self-Billing)** → das ist der **TypeCode 389**.

Factora trennt beide Fälle sauber über den TypeCode.

Rechnungsformate

Factora erzeugt elektronische Rechnungen nach der europäischen Norm **EN 16931**. Aus denselben Rechnungsdaten werden je nach Bedarf verschiedene Ausgabeformate generiert.

Unterstützte Formate

Format	Syntax	Datei	Einsatzzweck
XRechnung	CII (UN/CEFACT)	XML	Amtliches deutsches B2B-/B2G-Format
ZUGFeRD	CII, eingebettet	PDF/A-3	Menschen- und maschinenlesbare Hybrid-Rechnung
Peppol / UBL	UBL (OASIS)	XML	Internationaler Versand über das Peppol-Netzwerk
UBL CreditNote	UBL	XML	Korrekturrechnung im Peppol-Format

XRechnung (CII)

Die **XRechnung** ist der deutsche CIUS (Core Invoice Usage Specification) der EN 16931 und Pflichtformat für Rechnungen an öffentliche Auftraggeber (B2G). Factora erzeugt die XRechnung in **CII-Syntax** (UN/CEFACT Cross-Industry Invoice); Wurzelement ist **rsm:CrossIndustryInvoice**. Unterstützt werden u. a. Leitweg-ID (B2G), Reverse-Charge-Hinweise, Abschlüsse und Korrekturbezüge.

Als Spezifikationskennung (BT-24) wird standardmäßig gesetzt:

```
urn:cen.eu:en16931:2017#compliant#urn:xeinkauf.de:kosit:xrechnung_3.0
```

Als Geschäftsprozess (BT-23):

```
urn:fdc:peppol.eu:2017:poacc:billing:01:1.0
```

Ein im Payload explizit gesetztes **invoice_header.customization_id** überschreibt den Default.

Syntax-Auswahl. Standard ist CII (XRechnung). Auf UBL (Peppol) wird umgeschaltet, wenn eine Peppol-Customization explizit gesetzt ist oder der Käufer eine Peppol-Teilnehmerkennung trägt und keine Leitweg-ID (B2G) vorliegt.

ZUGFeRD (PDF/A-3)

ZUGFeRD ist eine Hybrid-Rechnung: ein optisch lesbares **PDF/A-3** mit **eingebetteter CII-XML**. Der Empfänger kann das PDF wie gewohnt ansehen, während Buchhaltungssysteme die strukturierten Daten direkt auslesen. Zusätzliche Anhänge (z. B. Lieferschein, Leistungsnachweis) können in das PDF/A-3 eingebettet werden.

PDF/A-3B und Factur-X-Kennzeichnung

Jedes erzeugte PDF ist ein **PDF/A-3B** und deklariert das im XMP-Paket (**pdfaid:part = 3**, **pdfaid:conformance = B**). Eingebettete Dateien werden als normgerechte *associated files* geführt (**/AF**, **/AFRelationship**).

Darauf liegt die **Factur-X-Kennzeichnung** als eigene Ebene. Sie wird nur geschrieben, wenn das Profil eine Conformance-Angabe festlegt — geraten wird nichts. Beide aktuellen Profile legen sie fest:

Profil	fx:ConformanceLevel	fx:Version	Eingebettete XML
XRechnung	XRECHNUNG	3.0	xrechnung.xml
EN 16931	EN 16931	1.0	factur-x.xml

Dazu kommt `fx:DocumentType = INVOICE` und `fx:DocumentFileName` passend zum Dateinamen.

Der Dateiname ist profilabhängig und normativ. ZUGFeRD 2.5.2 § 7.7 schreibt für das Referenzprofil XRECHNUNG `xrechnung.xml` vor und verbietet ausdrücklich, unter diesem Profil eine `factur-x.xml` einzubetten. Eine Hybrid-Rechnung ist bereits am falschen Dateinamen nicht konform.

Die Dokument-Metadaten (`/Subject`) tragen **bewusst keine Formatangabe**. Ein Format-Label gehört nur dorthin, wo die Struktur es auch trägt; prüfende Empfänger lesen ohnehin die PDF/A-3-Hülle und die `fx:*-XMP`-Einträge, nicht das DocInfo-Feld.

Inhaltlich gelten für die eingebettete XML dieselben Regeln wie für die XRechnung (siehe oben).

Peppol / UBL

Für den internationalen Versand über das **Peppol**-Netzwerk erzeugt Factora **UBL** (Universal Business Language) nach **Peppol BIS Billing 3.0**. Korrekturrechnungen werden im UBL-Format als **CreditNote** abgebildet.

Validierung & amtliche Prüfung (KoSIT)

Jede finalisierte Rechnung wird vor der Auslieferung gegen das amtliche Regelwerk geprüft. Factora nutzt dafür den offiziellen **KoSIT-Validator** (Koordinierungsstelle für IT-Standards) — denselben Prüfdienst, den auch die öffentliche Verwaltung einsetzt.

Format / Fall	Status
XRechnung 3.0 (CII)	Amtlich geprüft (KoSIT)
Peppol/UBL Standardrechnung (Billing 3.0)	Amtlich geprüft (Peppol-Validator)
Reverse Charge §13b (AE), inländisch — CII & UBL	Amtlich geprüft (KoSIT)
Self-Billing (389) — CII / XRechnung	Amtlich geprüft (KoSIT)
Self-Billing (389) — Peppol/UBL	Validiert — gegen das offizielle OpenPeppol-Schematron <i>Peppol BIS Self-Billing 3.0.1</i> ; die Prüfszenarien dafür pflegt Factora selbst, da weder KoSIT noch OpenPeppol eine fertige Validator-Konfiguration für Self-Billing ausliefert

Ergebnisse der Prüfung fließen in die Antwort-Hülle der API ein: KoSIT-Befunde erscheinen als strukturierte Fehler mit Regel (`rule`), Geschäftsbegriff (`bt`) und Fundstelle (`location`).

Format-Auswahl über die API

- Der **Atomic-Endpunkt** liefert die finalisierte Rechnung direkt als **ZUGFeRD-PDF** und **XRechnung-XML** (beides Base64) zurück.
- Zu einer gespeicherten Rechnung lassen sich die XML-Varianten gezielt abrufen: **CII/XRechnung** und **UBL/Peppol**.
- Der **Convert-Endpunkt** rendert EN-16931-Daten ohne Speicherung in geprüftes XML — nützlich zum Testen der eigenen Datenabbildung.

Details siehe Endpoints.

Steuerlogik

Factora bildet die umsatzsteuerliche Behandlung einer Rechnung normkonform nach EN 16931 ab. Jede Position trägt eine **Steuerkategorie (BT-151, UNTDID 5305)** und einen **Steuersatz (BT-152)**; daraus ergeben sich Pflichtangaben wie Befreiungsgründe und die dokumentweite Steueraufstellung.

Steuerkategorien

Code	Bezeichnung	Satz	Anwendung	Befreiungsgrund nötig
S	Regelsteuersatz	19 % oder 7 %	Normale Inlandsumsätze (auch ermäßigt besteuerte, 7 %)	nein
Z	Nullsatz	0 %	Nullsatz-Umsätze	nein
E	Steuerbefreit	0 %	u. a. §19 Kleinunternehmer	ja (Text)
AE	Reverse Charge	0 %	§13b UStG (Steuerschuld beim Empfänger)	ja (auto)
K	Innergemeinschaftliche Lieferung	0 %	§4 Nr. 1b UStG (EU → EU)	ja (auto)
G	Ausfuhrlieferung	0 %	§4 Nr. 1a UStG (Export Drittland)	ja (auto)
O	Nicht steuerbar	0 %	§3a Abs. 2 UStG (Leistung außerhalb)	individuell

Automatische Ableitung

Wird keine Kategorie ausdrücklich gesetzt, leitet Factora sie ab (erste zutreffende Regel gewinnt):

1. **Ausdrückliche Vorgabe** (z. B. **AE**, **K**, **G**) → wird übernommen.
2. **Kleinunternehmer-Profil** → **E**.
3. **Steuersatz > 0** → **S**.
4. **Steuersatz = 0** → **Z**.

Steuersätze

Unterstützt werden die deutschen Sätze **19 %**, **7 %** und **0 %**. Der Satz wird je Position im Feld **vat_rate** als Prozentwert übergeben (**19.00**, **7.00**, **0.00**). Für EU-B2C-Käufer (**customer_type**: "B2C", Käuferland ≠ Verkäuferland, EU) sind zusätzlich die Sätze des **Bestimmungslandes** gültig (OSS-Fernverkauf, §3c UStG — z. B. NL 21 %/9 %); die deutschen Sätze bleiben daneben erlaubt (unterhalb der 10.000-€-Schwelle wird weiter deutsch fakturiert).

Befreiungsgründe (BT-120 / BT-121)

Für befreite bzw. steuerfreie Kategorien sind ein **Befreiungstext (BT-120)** und ggf. ein **VATEX-Code (BT-121)** erforderlich. Factora füllt diese bei den deterministischen Fällen automatisch:

Kategorie	Befreiungstext (BT-120)	VATEX-Code (BT-121)
AE	„Steuerschuldnerschaft des Leistungsempfängers (§13b UStG / Art. 196 MwStSystRL)“	VATEX-EU-AE
K	„Innergemeinschaftliche Lieferung (§4 Nr. 1b UStG)“	VATEX-EU-IC
G	„Steuerfreie Ausfuhrlieferung (§4 Nr. 1a UStG)“	VATEX-EU-G
E (§19)	„Kleinunternehmer gemäß §19 UStG“	(<i>leer</i>)
O	individuell zu setzen	(<i>leer</i>)

Eigene Texte/Codes können je Position übergeben werden und haben Vorrang vor den Voreinstellungen.

Reverse Charge §13b (AE)

Die Steuerschuld geht auf den Leistungsempfänger über (z. B. Bauleistungen zwischen Unternehmern). Factora unterstützt **inländische und grenzüberschreitende** Reverse-Charge-Fälle.

Pflichtprüfungen:

- Käufer muss eine **USt-IdNr.** besitzen (BR-AE-02).
- Nur für **B2B/B2G**, nicht für Endverbraucher (B2C).
- Steuersatz muss **0 %** sein.
- Befreiungstext (BT-120) ist Pflicht (wird automatisch gesetzt).

Innergemeinschaftliche Lieferung (K) — §4 Nr. 1b

EU-grenzüberschreitende Lieferung an einen Unternehmer mit USt-IdNr.

Pflichtprüfungen:

- Käufer in einem **EU-Mitgliedstaat** (nicht im Inland).
- Käufer-**USt-IdNr.** vorhanden.
- **VIES-Prüfung** der USt-IdNr.: ungültig → blockiert; nicht erreichbar → nicht-blockierende Warnung mit späterer Wiederholung.
- Hinweis auf fehlenden **Gelangensnachweis** (§17a UStDV) als Warnung.

Ausfuhrlieferung (G) — §4 Nr. 1a

Lieferung an einen Empfänger **außerhalb der EU** (Drittland). Steuersatz 0 %, Befreiungstext automatisch.

Nicht steuerbar (O) — §3a Abs. 2

Leistung mit Ort außerhalb des deutschen Steuergebiets, an einen Empfänger im Drittland. Befreiungstext individuell.

Kleinunternehmer §19 UStG (E)

Bei Kleinunternehmer-Profil:

- Steuersatz **0 %**, Netto = Brutto (keine USt ausgewiesen).
- Kategorie **E**, Befreiungstext „Kleinunternehmer gemäß §19 UStG" ist Pflicht.
- VATEX-Code (BT-121) muss **leer** bleiben.
- Dokument-Hinweis: „Gemäß §19 UStG wird keine Umsatzsteuer berechnet."

Steuerlicher Vertreter (BG-11/12)

Tritt der Verkäufer über einen **Fiskalvertreter** auf (z. B. ausländischer Verkäufer mit Vertreter in DE), trägt die Rechnung dessen Angaben (BT-62 bis BT-69). Wird der Name gesetzt, werden USt-IdNr. und Land zu Pflichtfeldern. Die Angaben können tenantweit hinterlegt oder je API-Rechnung übergeben werden.

Grenzüberschreitende Einordnung

Aus Verkäufer-/Käuferland, USt-IdNr. und Kundentyp leitet Factora die zutreffende Behandlung ab:

- **Gleiches Land** → **S** (Regelsteuer).
- **EU-grenzüberschreitend, B2B mit USt-IdNr.** → **AE** (Reverse Charge).
- **EU-grenzüberschreitend, B2C** → **S**; gültig sind deutsche Sätze **und** die Sätze des Bestimmungslandes (OSS-Fernverkauf, §3c UStG).
- **Drittland-Export** → **G**.
- **Nordirland (XI, Brexit-Protokoll)**: Warenlieferungen EU-ähnlich (AE/S), Dienstleistungen als Export (**G**).

Dokumentweite Steueraufstellung (BG-23)

Zusätzlich zur positionsbezogenen Steuer (BG-27/28) erzeugt Factora die zusammengefasste **Steueraufstellung je Satz und Kategorie** (BG-23). Die Summen werden aus den Positionen exakt (Dezimal, keine Float-Rundung) berechnet und gegen die mitgelieferten Kontrollsummen (`validation_totals`) geprüft.

Prüfregeln — was eine Rechnung blockiert

Folgende Steuer-Prüfungen verhindern die Finalisierung (HTTP 400):

Fehlercode	Kategorie	Bedingung
<code>K_VAT_ID_MISSING</code>	K	Käufer ohne USt-IdNr.
<code>K_COUNTRY_NOT_EU</code>	K	Käufer nicht in der EU
<code>K_COUNTRY_DOMESTIC</code>	K	Käufer im selben Land wie Verkäufer
<code>K_VAT_ID_INVALID</code>	K	USt-IdNr. laut VIES ungültig
<code>AE_CUSTOMER_TYPE_B2C</code>	AE	Reverse Charge an Endverbraucher
<code>AE_VAT_ID_MISSING</code>	AE	Käufer ohne USt-IdNr. (BR-AE-02)
<code>AE_BT120_MISSING</code>	AE	Befreiungstext fehlt (BR-AE-10)
<code>G_COUNTRY_NOT_THIRD</code>	G	Käufer innerhalb der EU
<code>O_COUNTRY_NOT_THIRD</code>	O	Käufer innerhalb der EU
<code>E_KU_BT120_MISSING</code>	E	Kleinunternehmer ohne Befreiungstext
<code>E_KU_BT121_SET</code>	E	Kleinunternehmer mit gesetztem VATEX-Code

Nicht-blockierende Warnungen (HTTP 200, protokolliert):

Warncode	Kategorie	Bedingung
<code>K_VAT_ID_UNCHECKED</code>	K	VIES-Dienst nicht erreichbar
<code>K_DELIVERY_PROOF_MISSING</code>	K	Gelangensnachweis noch nicht erfasst

Einfrieren bei Finalisierung (GoBD)

Steuerkategorie, Satz und Befreiungsgründe werden bei der Finalisierung als Snapshot **eingefroren**, damit XML und PDF jederzeit reproduzierbar bleiben. Siehe Compliance & Archivierung.

Compliance & Archivierung

Factora ist auf die deutschen Aufbewahrungs- und Nachvollziehbarkeitsanforderungen (GoBD) ausgelegt. Finalisierte Rechnungen sind unveränderlich, werden verschlüsselt archiviert und jede Statusänderung wird protokolliert.

Unveränderlichkeit finalisierter Rechnungen (GoBD)

Sobald eine Rechnung finalisiert ist, sind ihre rechnungsrelevanten Daten gesperrt. Geschützte (eingefrorene) Felder sind u. a.:

- Rechnungsnummer, Rechnungsdatum, Fälligkeitsdatum
- Käufer/Empfänger-Snapshot
- Netto-, Steuer- und Bruttobeträge
- Leitweg-ID (B2G)
- Steuerkategorie, Steuersatz, Befreiungsgründe je Position

Ein Schreibversuch auf eine finalisierte Rechnung wird abgelehnt. Erlaubt sind nur definierte **Statusübergänge** (z. B. finalisiert → versendet → bezahlt → storniert).

Löschen ist ausschließlich im Entwurfsstatus möglich. Eine finalisierte Rechnung kann nicht gelöscht, sondern nur über eine Korrekturrechnung berichtigt werden.

Verschlüsseltes Archiv

Bei der Finalisierung wird die Rechnung in einem verschlüsselten Archiv abgelegt. Dabei werden die Metadaten (Nummer, Datum, Empfänger, Positionen, Beträge) erfasst und über einen **SHA-256-Integritätshash** gesichert, sodass nachträgliche Veränderungen erkennbar wären. Der Archivstatus jeder Rechnung ist nachvollziehbar (ausstehend / archiviert / fehlgeschlagen).

Änderungsprotokoll (Audit-Log)

Jede Statusänderung einer Rechnung — insbesondere der Übergang vom Entwurf zur finalisierten Rechnung — wird mit Zeitpunkt und auslösendem Akteur in einem Änderungsprotokoll festgehalten.

Normprüfung (EN 16931 / KoSIT)

Vor der Auslieferung durchläuft jede Rechnung die fachliche Prüfung gegen das EN-16931-Regelwerk und — je nach Format — den amtlichen **KoSIT-Validator**. Befunde werden strukturiert zurückgegeben (Regel, Geschäftsbegriff, Fundstelle). Welche Formate amtlich abgenommen sind, steht unter Rechnungsformate.

Datenschutz & Betrieb

- Betrieb und Datenhaltung in **Deutschland**.
- **DSGVO**-konform mit Auftragsverarbeitungsvertrag (AVV).
- Tägliche Sicherung der Daten.
- Sensible Felder (z. B. Bankverbindung) werden verschlüsselt gespeichert.

Datenexport

- Vollständiger Konto-Datenexport auf Anforderung.
- **DATEV-Export** der Buchungsdaten für die Finanzbuchhaltung — siehe Exporte.

API-Überblick

Die öffentliche **Factora API v1** ist eine REST-Schnittstelle zum Erzeugen, Prüfen, Abrufen und Versenden elektronischer Rechnungen aus einem Fremdsystem heraus.

- **Basis-URL:** <https://console.factora.software/api/v1>
- **Datenformat:** JSON (UTF-8)
- **Spezifikation:** OpenAPI 3.1 — interaktiv unter <https://console.factora.software/api/docs/>

Authentifizierung

Jeder Aufruf wird mit einem **API-Schlüssel** als Bearer-Token autorisiert:

```
Authorization: Bearer fa_live_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

API-Schlüssel

- **Live-Schlüssel** beginnen mit **fa_live_**, **Sandbox-Schlüssel** mit **fa_test_**. Ältere Schlüssel ohne Umgebungsmarker (**fa_...**) authentifizieren weiterhin unverändert.
- Die ersten 14 Zeichen (Präfix + 6 Zeichen) bilden den **Key-Präfix**, der in Listen und Logs der Console sichtbar ist und den Schlüssel identifiziert, ohne das Geheimnis preiszugeben.
- Der Klartext wird **nur einmal** bei der Erstellung angezeigt. Serverseitig liegt kein Klartext: gespeichert werden ein **SHA-256-Hash** und ein **HMAC-SHA256**. Geht ein Schlüssel verloren, gibt es kein Recovery — neuen erzeugen, alten deaktivieren.
- Optional pro Schlüssel: **Ablaufdatum**, **IP-Whitelist** und **Scopes**.
- **Ablaufdatum** ist optional; ohne Wert läuft der Schlüssel nie ab. Nach Ablauf antwortet die API mit **401**.
- **Rotation:** In der Console lässt sich ein Schlüssel gegen einen neuen ersetzen. Der alte wird deaktiviert, bleibt aber für die Audit-Historie erhalten.
- Verwaltung in der Console unter <https://console.factora.software/dashboard/api-keys>.

IP-Whitelist

Pro Schlüssel lässt sich eine Liste erlaubter IP-Adressen hinterlegen. Eine **leere Liste = alle IPs erlaubt**. Ist die Liste gefüllt, muss der Request von einer **exakt gelisteten Einzel-IP** kommen — **CIDR-Bereiche werden bewusst abgelehnt**, weil die Prüfung serverseitig auf exakte Übereinstimmung geht. Maßgeblich ist die vom Reverse-Proxy gesetzte Client-IP.

Scopes (Berechtigungen)

Ein Schlüssel kann auf bestimmte Rechte beschränkt werden. Ohne Einschränkung (leere Liste) gelten alle Rechte. Die Prüfung ist **methodenbasiert**: lesende Methoden (**GET**) verlangen den **...:read**-Scope, schreibende (**POST** / **PATCH** / **DELETE**) den **...:write**-Scope.

Scope	Bedeutung
invoices:read	Rechnungen lesen, PDF/XML herunterladen
invoices:write	Rechnungen erstellen, finalisieren, versenden
customers:read	Kunden lesen
customers:write	Kunden anlegen
mandanten:read	Mandanten lesen
mandanten:write	Mandanten anlegen/ändern/löschen
exports:read	DATEV-Export herunterladen
webhooks:read	Webhooks und Zustellungen lesen
webhooks:write	Webhooks verwalten, Test senden

Sandbox vs. Live

Mit einem **Sandbox-Schlüssel** lässt sich die Anbindung kostenlos und folgenlos testen. Es durchläuft dieselbe Pipeline wie Live, aber nichts wird gespeichert.

Aspekt	Sandbox	Live
Speicherung	keine (Trockenlauf)	vollständig
Erreichbare Endpunkte	nur Atomic + Hilfsendpunkte (Preview/Convert)	alle
Limits/Kontingent	übersprungen	angewendet (402 bei Limit, 409 bei Dublette)
Idempotenz	optional	erforderlich
Antwort-Status	200 mit <code>meta.sandbox = true</code>	201 bei Anlage

Idempotenz

Schreibende Aufrufe (Atomic Invoice) erfordern im Live-Betrieb einen Idempotenz-Schlüssel, damit ein versehentlich doppelt gesendeter Aufruf nicht zwei Rechnungen erzeugt:

```
Idempotency-Key: 550e8400-e29b-41d4-a716-446655440000
```

- **Format:** UUID.
- **Gültigkeit:** 48 Stunden.
- **Wiederholung:** Ein erneuter Aufruf mit demselben Schlüssel liefert die zwischengespeicherte Antwort zurück, gekennzeichnet mit dem Header `Idempotent-Replayed: true`.
- Im Sandbox-Modus ist der Schlüssel optional (es wird ohnehin nichts gespeichert).

Limits (Throttling)

Es gibt zwei voneinander unabhängige Mengensteuerungen.

Tageslimit (Aufrufe / 24 h)

Gilt **pro Schlüssel** und ist ein reiner **Anti-Missbrauchs-Schutz**, **keine Preisstufe** — der API-Tarif wird nutzungsbasiert abgerechnet.

Tarif	Aufrufe / 24 h	Schlüssel max.
api	50.000	20

Wird das Limit überschritten, antwortet die API mit **HTTP 429** und einem **Retry-After**-Header (Sekunden bis zum Reset). Das Fenster ist gleitend, es gibt keinen Reset zu einer festen Uhrzeit. **Sandbox-Aufrufe zählen nicht mit.** Begründete Ausreißer werden manuell angehoben.

Weiche Dokument-Quota

Zusätzlich trägt jede schreibende Antwort den Stand der monatlichen Dokument-Quota mit — **rein informativ, sie blockt einen Request nie:**

- in `meta.quota` mit `used`, `limit`, `percent`, `level`
- als Header `X-Factora-Documents-Used` und `X-Factora-Documents-Limit`

Level	Auslastung	Verhalten
<code>ok</code>	< 80 %	normal
<code>warning</code>	80–99 %	Hinweis, kein Block
<code>exceeded</code>	≥ 100 %	Hinweis, kein Block

Im API-Tarif ist die monatliche Dokumentgrenze **unbegrenzt**; abgerechnet wird nutzungsbasiert.

Harte Rechnungsgrenze (nur Free)

Davon zu unterscheiden ist die harte Monatsgrenze des Free-Tarifs (5 Rechnungen/Monat). Wird sie erreicht, blockt die API das Erstellen mit **HTTP 402** (`limit_reached`). Solo-, Business- und API-Tarif haben hier keine harte Grenze.

Fehler- und Antwortstruktur

Alle Antworten verwenden eine einheitliche Hülle:

```
{
  "valid": true,
  "data": { },
  "errors": [],
  "meta": {}
}
```

Im Fehlerfall:

```
{
  "valid": false,
  "data": null,
  "errors": [
    {
      "code": "schema",
      "severity": "error",
      "message": "Beschreibung des Fehlers",
      "field": "items[0].vat_rate",
      "rule": "BR-C0-3",
      "bt": "BT-152",
      "location": "//ram:ApplicableTradeTax"
    }
  ],
  "meta": {}
}
```

Die Felder der Hülle:

Feld	Inhalt
<code>valid</code>	<code>true</code> bei Erfolg, <code>false</code> im Fehlerfall
<code>data</code>	das Ergebnis (Objekt oder Array), <code>null</code> im Fehlerfall
<code>errors</code>	Liste der Fehler-Einträge, bei Erfolg leer
<code>meta</code>	Zusatzinfos (Paginierung, Quota, <code>sandbox</code> -Flag)

Das **Fehlerobjekt** enthält **code**, **severity** und **message** immer, die übrigen Felder je nach Fehlerart: **field** (Eingabepfad), **rule** (EN-16931-/Geschäftsregel), **bt** (Geschäftsbegriff) und **location** (XPath bei XML-Befunden).

Validierungsbefunde (**code**: "kosit" oder "business") führen **rule**, **bt** und **location** immer — **null**, wenn der Prüfbericht sie nicht benennt (etwa bei reinen Schema-Fehlern). Ein Client kann diese Schlüssel dort also ohne Existenzprüfung auslesen. Bei allen anderen Fehlerarten (Authentifizierung, Kontingent, Schema-Validierung der Eingabe) fehlen sie weiterhin ganz.

Bei **5xx** greift die Hülle nicht — das ist ein unerwarteter Serverfehler. Bitte mit Request-Details an den Support melden.

Erfolgs-Codes

HTTP	Bedeutung
200	Erfolg (Lesen, Idempotenz-Replay, Sandbox-Trockenlauf, Preview)
201	Erstellt (Live-Rechnung, Kunde, Mandant, Webhook)
204	Kein Inhalt (erfolgreiches DELETE)

Fehlercodes

HTTP	Code	Bedeutung
400	schema	Ungültiges JSON, fehlende/falsche Felder, Summen-Abweichung, ungültiger Idempotenz-Schlüssel
400	business	Verstoß gegen EN-16931-/GoBD-Geschäftsregel
400	kosit	Befund der amtlichen KoSIT-Prüfung
400	invalid_state	Rechnung im falschen Status (z. B. Versand eines Entwurfs)
400	unsupported_format	Eingabeformat nicht unterstützt
401	authentication	Schlüssel fehlt/ungültig/inaktiv/abgelaufen, IP nicht erlaubt
402	limit_reached	Harte Rechnungs-Obergrenze erreicht — nur Free-Tarif
403	permission	Kein API-Zugang, fehlender Scope, Sandbox-Schlüssel auf unzulässigem Endpunkt, IP nicht erlaubt
404	not_found	Objekt existiert nicht oder gehört zu anderem Konto
405	method_not_allowed	HTTP-Methode am Endpunkt nicht erlaubt
406	not_acceptable	Nicht unterstütztes Antwortformat
409	conflict	Doppelte Rechnungsnummer, Dublette, Mandant in Verwendung
415	unsupported_media_type	Nicht unterstützter Content-Type
429	rate_limited	Tageskontingent überschritten — Antwort trägt Retry-After
502	bad_gateway	Webhook-Zustellung fehlgeschlagen (Test)

Mandanten (Multi-Mandanten-Modell)

Ein Integrator kann unter einem Konto beliebig viele **Mandanten** (eigene Endkunden) führen. Jeder Mandant hat eigene Verkäufer-Stammdaten. Ein Aufruf wählt den Mandanten über `mandant_id` im Rechnungs-Payload; dessen Stammdaten werden automatisch als Verkäufer eingesetzt. Kunden und Rechnungen werden **pro Mandant getrennt** geführt. Siehe Endpoints und Atomic Invoice.

Endpoints

Alle Pfade relativ zur Basis-URL <https://console.factora.software/api/v1>. Authentifizierung, Antwort-Hülle und Fehlercodes siehe API-Überblick.

Schema & interaktive Referenz

Die API ist vollständig per **OpenAPI 3.1** beschrieben (Titel „Factora API“, Version 1.0.0). Bei Abweichungen zwischen Prosa und Schema ist das **Schema maßgeblich**.

Ressource	Pfad	Zugriff
OpenAPI-Schema (3.1)	GET /api/schema/	öffentlich
Swagger-UI	GET /api/docs/	öffentlich
ReDoc	GET /api/redoc/	öffentlich
Scalar-Referenz	https://console.factora.software/api/v1/docs/	öffentlich
Vollschema	GET /api/schema/full/ · GET /api/docs/full/	authentifiziert

Vollständige Endpunktliste

Methode	Pfad	Zweck	Scope	Sandbox
POST	/invoices/atomic/	Rechnung in einem Aufruf erstellen, finalisieren, PDF/XML zurückgeben	invoices:write	✓
POST	/invoices/preview/	Summen live berechnen (ohne Speicherung)	invoices:read	✓
POST	/invoices/convert/	EN-16933-JSON → geprüftes XML (ohne Speicherung)	invoices:write	✓
GET	/invoices/	Rechnungen auflisten (paginiert, filterbar)	invoices:read	—
POST	/invoices/	Rechnung als Entwurf anlegen	invoices:write	—
GET	/invoices/{id}/	Einzelne Rechnung abrufen	invoices:read	—
POST	/invoices/{id}/finalise/	Entwurf finalisieren (PDF/XML erzeugen)	invoices:write	—
POST	/invoices/{id}/send/	Finalisierte Rechnung per E-Mail versenden	invoices:write	—
GET	/invoices/{id}/pdf/	ZUGFeRD-PDF herunterladen	invoices:read	—
GET	/invoices/{id}/xml/	XRechnung XML herunterladen	invoices:read	—
GET	/invoices/{id}/xml/CEB/	XRechnung XML live erzeugen	invoices:read	—
GET	/invoices/{id}/xml/ABL/Peppol/	ABL/Peppol XML live erzeugen	invoices:read	—
POST	/invoices/{id}/dispatch/	Rechnung zum Transport einmehren (E-Mail/Peppol)	invoices:write	—
GET	/invoices/{id}/dispatch/	Verbindungsadresse abrufen	invoices:read	—
GET	/customers/	Kunden auflisten	customers:read	—
POST	/customers/	Kunden anlegen	customers:write	—
GET	/mandanten/	Mandanten auflisten	mandanten:read	—
POST	/mandanten/	Mandant anlegen	mandanten:write	—
GET	/mandanten/{id}/	Mandant abrufen	mandanten:read	—
PATCH	/mandanten/{id}/	Mandant ändern (partiell)	mandanten:write	—
DELETE	/mandanten/{id}/	Mandant löschen (nur ohne Rechnungen/Kunden)	mandanten:write	—
POST	/mandanten/bulk/	Bis zu 200 Mandanten anlegen (alles-oder-nichts)	mandanten:write	—
GET	/exports/daten/	DATeV-Buchungsdaten als CSV	exports:read	—
GET	/webhooks/	Webhooks auflisten	webhooks:read	—
POST	/webhooks/	Webhook anlegen	webhooks:write	—
GET	/webhooks/{id}/	Webhook abrufen	webhooks:read	—
PATCH	/webhooks/{id}/	Webhook ändern	webhooks:write	—
DELETE	/webhooks/{id}/	Webhook deaktivieren	webhooks:write	—
POST	/webhooks/{id}/test/	Test-Webhook senden	webhooks:write	—
GET	/webhooks/{id}/details/	Detaillierte Auflistung eines Webhooks	webhooks:read	—

Rechnungen

Atomic Invoice — **POST** `/invoices/atomic/`

Der zentrale Endpunkt für die ERP-Integration: ein einziger Aufruf erstellt, finalisiert und prüft die Rechnung und liefert das fertige **ZUGFeRD-PDF** und die **XRechnung-XML** (beide Base64) zurück. Erfordert im Live-Betrieb einen Idempotenz-Schlüssel. Das **vollständige Feldschema** steht unter Atomic Invoice.

Preview — **POST** `/invoices/preview/`

Berechnet aus übergebenen Positionen die Summen exakt (Dezimal) inklusive Aufschlüsselung je Steuersatz. Keine Speicherung, auch mit Sandbox-Schlüssel und bei inaktivem Abo nutzbar.

Convert — **POST** `/invoices/convert/`

Rendert EN-16931-Rechnungsdaten ohne Speicherung in geprüftes XML (CII/XRechnung oder UBL, je nach Rechnungstyp) und gibt KoSIT-Befunde zurück. Ideal, um die eigene Datenabbildung zu testen.

Klassischer Lebenszyklus

Statt des Atomic-Wegs lässt sich eine Rechnung auch schrittweise führen: anlegen (**POST** `/invoices/`) → finalisieren (`/finalize/`) → herunterladen (`/pdf/`, `/xml/`) → versenden (`/send/`) oder einreihen (`/dispatch/`). Listen sind paginiert und filterbar; per **mandant**-Filter lässt sich auf einen Mandanten einschränken.

Mandanten

CRUD über `/mandanten/`. Der **Bulk-Endpunkt** legt bis zu 200 Mandanten in einer Transaktion an (**alles oder nichts** — schlägt eine Zeile fehl, z. B. durch einen doppelten Namen, wird nichts angelegt; Antwort 409). Ein Mandant kann nicht gelöscht werden, solange Rechnungen oder Kunden an ihm hängen (409, GoBD).

Kunden

Auflisten und Anlegen über `/customers/`. Auf dem Mandanten-Pfad werden Kunden pro Mandant getrennt geführt.

Exporte

GET `/exports/datev/` — siehe Exporte.

Webhooks

CRUD, Testversand und Zustellhistorie über `/webhooks/` — siehe Webhooks.

Atomic Invoice — vollständiges Feldschema

POST /api/v1/invoices/atomic/

Erstellt, finalisiert und prüft eine Rechnung in einem einzigen Aufruf und liefert das fertige ZUGFeRD-PDF und die XRechnung-XML zurück. Dieses Dokument listet **jedes Feld** des Payloads mit Typ, Pflicht/Optional, Default, Format und EN-16931-Bezug (BT-Codes).

Konventionen: Beträge sind Dezimal-Strings (2 Nachkommastellen), Mengen 3 Stellen, Sätze 2 Stellen; Datumsangaben ISO 8601 (YYYY-MM-DD).

Aufbau des Payloads

Feld	Typ	Pflicht	Default	Bemerkung
api_mode	string	ja	—	muss "atomic_single_post" sein
invoice_header	Objekt	ja	—	Rechnungskopf (s. u.)
validation_totals	Objekt	ja	—	Kontrollsummen aus dem Quellsystem
seller_snapshot	Objekt	bedingt*	—	Verkäufer-Stammdaten
mandant_id	integer	bedingt*	—	alternativ: Mandant referenzieren
buyer	Objekt	ja	—	Käufer/Empfänger
shipping	Objekt	nein	—	abweichende Lieferadresse
items	Array	ja	—	min. 1 Position
attachments	Array	nein	[]	bis 10 Anhänge, max. 15 MB gesamt
extended_fields	Objekt	nein	{}	optionale EN-16931-Zusatzfelder

* **Genau eines** von `seller_snapshot` oder `mandant_id` ist anzugeben. Bei `mandant_id` werden die Verkäuferdaten automatisch aus dem Mandanten gesetzt.

Rechnungskopf — invoice_header

[illegible]

Kontrollsummen — validation_totals

Feld	Typ	Pflicht	Bemerkung
net_amount	decimal	ja	Summe der Netto-Positionsbeträge
vat_amount	decimal	ja	Gesamte USt
gross_amount	decimal	ja	Brutto (Netto + USt)

Der Server berechnet die Summen aus den Positionen neu und vergleicht sie mit diesen Werten. **Toleranz: 0,02 € je Feld** (netto, USt., brutto), absolut gerechnet. Darüber wird die Anfrage mit **HTTP 400** abgelehnt; die Meldung nennt den berechneten und den gesendeten Wert.

Innerhalb der Toleranz wird der Beleg angenommen, und es gelten die **vom Server berechneten** Summen — nicht die gesendeten. Weichen beide voneinander ab, weist die Antwort das in `meta.totals_adjusted` aus, damit die Differenz nachvollziehbar dokumentiert werden kann:

```
"meta": {
  "totals_adjusted": {
    "tolerance": "0.02",
    "fields": {
      "vat_amount": { "sent": "0.47", "used": "0.48", "delta": "0.01" },
      "gross_amount": { "sent": "2.97", "used": "2.98", "delta": "0.01" }
    }
  }
}
```

`delta` ist `used` - `sent`. Stimmen die Summen auf den Cent überein, fehlt der Schlüssel `totals_adjusted` ganz. Der häufigste Grund für eine Differenz ist eine andere Rundungsregel im Quellsystem: Factora rundet kaufmännisch je Position auf zwei Nachkommastellen und summiert danach.

Verkäufer — seller_snapshot

Feld	Typ	Pflicht	Default	BT	Bemerkung
name	string(255)	ja	—	BT-27	Firmenname
street	string(255)	ja	—	BT-31	Straße
zip	string(20)	ja	—	BT-34	PLZ
city	string(255)	ja	—	BT-35	Ort
country	string(2)	nein	"DE"	BT-40	ISO 3166-1
country_subdivision	string(100)	nein	" "	BT-39	Bundesland/Region
address_line2	string(255)	nein	" "	BT-36	Adresszusatz
address_line3	string(255)	nein	" "	BT-162	Adresszusatz
vat_id	string	nein	" "	BT-31	USt-IdNr.
tax_number	string	nein	" "	—	Steuernummer
gln	string	nein	" "	—	GLN (13 Stellen, GS1-Prüfziffer)
identifizier	string(64)	nein	" "	BT-29	weitere Verkäufererkennung
trading_name	string(255)	nein	" "	BT-28	Handelsname
legal_registration_number	string(64)	nein	" "	BT-30	Handelsregisternummer
additional_legal_info	string(255)	nein	" "	BT-33	rechtliche Zusatzinfo
iban	string	nein	" "	BT-84	Zahlungs-IBAN
bic	string	nein	" "	—	BIC/SWIFT
contact	Objekt	nein	—	BG-6	Ansprechpartner (s. u.)
tax_representation	Objekt	nein	—	BG-11/12	steuerlicher Vertreter (s. u.)

Ansprechpartner — **contact**

Feld	Typ	Pflicht	Bemerkung
name	string(255)	nein	Kontaktperson
phone	string(50)	nein	Telefon
email	string(255)	nein	E-Mail

Steuerlicher Vertreter — **tax_representative** (BG-11/12)

Alle Felder optional; wird **name** gesetzt, werden **vat_id** und **country** zu Pflichtangaben.

Feld	Typ	BT	Bemerkung
name	string(255)	BT-62	Name des Vertreters
vat_id	string(64)	BT-63	USt-IdNr. des Vertreters
street	string(255)	BT-64	Straße
address_line2	string(255)	BT-65	Adresszusatz
address_line3	string(255)	BT-164	Adresszusatz
zip	string(20)	BT-67	PLZ
city	string(255)	BT-66	Ort
country_subdivision	string(100)	BT-68	Region
country	string(2)	BT-69	Land

Käufer — **buyer**

Feld	Typ	Pflicht	Default	BT	Bemerkung
name	string(255)	ja	—	BT-44	Firmenname
street	string(255)	ja	—	BT-49	Straße
zip	string(20)	ja	—	BT-50	PLZ
city	string(255)	ja	—	BT-50	Ort
country	string(2)	nein	"DE"	BT-55	ISO 3166-1
country_subdivision	string(100)	nein	""	BT-54	Bundesland/Region
address_line2	string(255)	nein	""	BT-51	Adresszusatz
address_line3	string(255)	nein	""	BT-163	Adresszusatz
vat_id	string	nein	""	BT-48	USt-IdNr. (Pflicht bei AE/K)
gln	string	nein	""	BT-44	GLN (13 Stellen, GS1-Prüfziffer)
identifier	string(64)	nein	""	BT-46	weitere Käuferkennung
debtor_number	string(20)	nein	""	—	DATEV-Debitorennummer (nur Export, nicht im XML)
customer_number	string(50)	nein	""	—	Ihre eigene Kundennummer (siehe unten; nur PDF, nicht im XML)
branch_id	string	nein	""	—	Filial-/Standortkennung
trading_name	string(255)	nein	""	BT-45	Handelsname
legal_registration	string(64)	nein	""	BT-47	Handelsregisternummer
contact	Objekt	nein	—	BG-9	Ansprechpartner (wie oben)

Kundennummer — `buyer.customer_number`

Die Nummer, unter der der Käufer in **Ihrem** System geführt wird. Sie erscheint auf dem PDF als „Kundennummer“ und steht nicht im XML (EN 16931 kennt dafür kein Feld — für eine Käuferkennung im XML nutzen Sie `gln` (BT-44) oder `identifizier` (BT-46)).

Die Nummer ist zugleich die **Identität des Kunden**: Rechnungen mit derselben Kundennummer landen auf demselben Kundendatensatz, auch wenn sich der Firmenname zwischenzeitlich geändert hat. Umgekehrt gilt: ändern Sie die Nummer eines bestehenden Kunden, übernehmen wir das — bereits ausgestellte Rechnungen behalten die Nummer, die zum Zeitpunkt der Ausstellung galt.

Eindeutig ist sie **je Mandant**: zwei Mandanten dürfen dieselbe Nummer vergeben, innerhalb eines Mandanten bezeichnet sie genau einen Kunden.

Lassen Sie das Feld weg, vergibt Factora weiterhin selbst eine Nummer aus dem Firmennamen (`API-...`). Schicken Sie die Nummer für einen Käufer, den wir bereits unter einer solchen `API-...`-Nummer führen, ersetzt Ihre Nummer die generierte — es entsteht kein zweiter Datensatz.

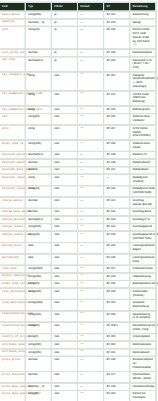
Lieferadresse — `shipping` (BG-13/15)

Optional; fehlt sie, gilt die Käuferadresse. Wird sie angegeben, sind `name`, `street`, `zip`, `city` Pflicht.

Feld	Typ	Pflicht	Default	BT
<code>name</code>	string(255)	ja	—	BT-71
<code>street</code>	string(255)	ja	—	BT-73
<code>zip</code>	string(20)	ja	—	BT-74
<code>city</code>	string(255)	ja	—	BT-75
<code>country</code>	string(2)	nein	"DE"	BT-80
<code>country_subdivision</code>	string(100)	nein	""	BT-79
<code>address_line2</code>	string(255)	nein	""	BT-76
<code>address_line3</code>	string(255)	nein	""	BT-165
<code>gln</code>	string	nein	""	—

Positionen — `items[]`

Pflichtfelder je Position: `description`, `quantity`, `unit`, `unit_price_net`, `vat_rate`.



Anhänge — attachments[]

Bis zu **10 eingebettete Anhänge**, **max. 15 MB** je Datei und gesamt. Pro Anhang ist **entweder** `content_base64` **oder** `external_url` anzugeben. Der MIME-Typ wird gegen eine Whitelist geprüft und der Dateiinhalt zusätzlich anhand der „Magic Bytes“ verifiziert.

Erlaubte MIME-Typen: PDF, PNG, JPEG, CSV, ODS (OpenDocument), XLSX, XLS.

Feld	Typ	Pflicht	Default	BT	Bemerkung
<code>filename</code>	string(255)	ja	—	—	Dateiname
<code>mime_type</code>	string(100)	ja	—	BT-125-1	aus Whitelist
<code>type</code>	string	nein	"OTHER"	—	INVOICE_IMAGE / DELIVERY_NOTE / OTHER
<code>content_base64</code>	string	bedingt	""	BT-125	eingebettete Datei (XOR <code>external_url</code>)
<code>external_url</code>	string(500)	bedingt	""	BT-124	externer Link (XOR <code>content_base64</code>)
<code>document_reference</code>	string(200)	nein	""	BT-122	Dokumentkennung
<code>description</code>	string(255)	nein	""	BT-123	Beschreibung
<code>delivery_note_number</code>	string	nein	""	—	bei <code>type=DELIVERY_NOTE</code>

Zusatzfelder — extended_fields

Frei verwendbares Objekt für weitere EN-16931-Felder. Werte aus dem Aufruf überschreiben etwaige am API-Schlüssel hinterlegte Voreinstellungen.

Antwort

Erfolg (Live): HTTP 201. **Sandbox:** HTTP 200 mit `meta.sandbox = true`.

```
{
  "valid": true,
  "data": {
    "id": 12345,
    "invoice_number": "RE-2026-001",
    "status": "final",
    "total": "1190.00",
    "pdf_base64": "JVBERi0xLjQK...",
    "xml_base64": "PD94bWwgdmVy..."
  },
  "errors": [],
  "meta": {}
}
```

Fehlerfälle und Fehlercodes: siehe API-Überblick.

Wichtige Kombinationsregeln

1. Genau eines von `seller_snapshot` / `mandant_id`.
2. Bei befreiten Kategorien (E, AE, K, G, O) Befreiungstext/-code angeben (AE/K/G werden automatisch gefüllt).
3. AE/K erfordern eine Käufer-USt-IdNr.; AE nur für B2B/B2G.
4. `validation_totals` müssen zur Positionsberechnung passen.
5. GLN/GTIN, falls angegeben, müssen gültige GS1-Prüfziffern haben.
6. Anhänge: `content_base64` XOR `external_url`.
7. `prepaid_amount` (BT-113) muss $\leq$ Bruttobetrag sein, sonst 400.

Bereits beglichene Belege

Läuft die Zahlung außerhalb des Belegs (z. B. Plattform-Abrechnung, Gutschriftverfahren nach §14 Abs. 2 UStG), ist bei Erstellung nichts mehr fällig. Empfohlene Kombination:

- `invoice_header.prepaid_amount` = Bruttobetrag → XML trägt BT-113 und BT-115 = `0.00` (kein offener Posten beim Empfänger-Import); das PDF zeigt „Bereits gezahlt“ in der Summenübersicht und ersetzt Bankblock, Zahlungs-QR und Fälligkeitszeile durch einen Beglichen-Hinweis.
- `payment_means_code` **nicht** 58/59 setzen — z. B. `1` (Instrument not defined), `68` (Online payment service) oder `97` (Clearing between partners). BR-DE-23-a erzwingt nur bei `58` eine IBAN (BG-17).
- `iban/bic` im `seller_snapshot` weglassen, wenn kein Konto auf dem Beleg erscheinen soll (Achtung: Tenant-Stammdaten füllen Snapshot-Lücken — BG-17 entfällt nur, wenn beide keine IBAN tragen).
- Optional `payment_terms` mit eigenem Hinweistext (BT-20), sonst rendert das PDF den Standard-Beglichen-Hinweis.

Webhooks

Webhooks benachrichtigen ein Fremdsystem in Echtzeit über Ereignisse zu Rechnungen. Factora sendet dazu einen signierten HTTP-POST an eine von Ihnen hinterlegte HTTPS-URL.

Verwaltung über die Endpunkte unter </api/v1/webhooks/> (siehe Endpoints).

Ereignistypen

Ereignis	Auslöser
<code>invoice.created</code>	Rechnungsentwurf angelegt
<code>invoice.finalized</code>	Rechnung finalisiert (PDF/XML erzeugt)
<code>invoice.dispatched</code>	Rechnung zum Transport eingereicht
<code>invoice.delivered</code>	Rechnung erfolgreich zugestellt (E-Mail/Peppol)
<code>invoice.paid</code>	Rechnung als bezahlt markiert
<code>invoice.cancelled</code>	Rechnung storniert
<code>invoice.rejected</code>	Rechnung abgelehnt (Prüf-/Zustellfehler)

Beim Anlegen eines Webhooks wählen Sie über `subscribed_events` die abonnierten Ereignisse. Zum Testen existiert zusätzlich `webhook.test` (siehe unten).

Payload

Der Rumpf ist ein **flaches JSON-Objekt** — ein Schnappschuss der Rechnung zum Zeitpunkt des Ereignisses:

Feld	Inhalt
<code>event_type</code>	Ereignistyp (siehe oben)
<code>invoice_number</code>	Rechnungsnummer
<code>invoice_id</code>	interne ID
<code>status</code>	Rechnungsstatus
<code>invoice_type</code>	Rechnungstyp
<code>total / subtotal / tax_amount</code>	Beträge (String, Dezimal)
<code>currency</code>	Währungscode
<code>customer_name</code>	Kundenname
<code>invoice_date / due_date</code>	Rechnungs- und Fälligkeitsdatum (ISO)
<code>occurred_at</code>	Zeitstempel des Ereignisses (ISO 8601)

Beispiel (`invoice.finalized`):

```
{
  "event_type": "invoice.finalized",
  "invoice_number": "RE-2026-0001",
  "invoice_id": 42,
  "status": "finalized",
  "total": "595.00",
  "currency": "EUR",
  "occurred_at": "2026-06-18T10:15:00+00:00"
}
```

Aufbau der Zustellung

Jeder Webhook-Request enthält:

Header	Wert
Content-Type	application/json
X-Factora-Event	Ereignistyp (z. B. <code>invoice.finalized</code>)
X-Factora-Signature	sha256=<hex> — HMAC-SHA256 des Rumpfes
User-Agent	Factora-Webhooks/1.0

Signaturprüfung

Die Signatur sichert Echtheit und Unverändertheit. Berechnung:

```
X-Factora-Signature = "sha256=" + HMAC_SHA256(secret, raw_request_body)
```

Der Empfänger berechnet denselben HMAC mit dem **bei der Webhook-Anlage vereinbarten Secret** über den **rohen Request-Body** und vergleicht das Ergebnis. Stimmen die Werte nicht überein, ist der Request abzulehnen.

Das Secret wird auf Ihrer Seite gewählt und bei Factora **AES-256-GCM**- verschlüsselt gespeichert. Es verlässt das System nicht im Klartext.

Zustellung & Wiederholung

Eigenschaft	Wert
Erfolgskriterium	HTTP-Status 200–299
Timeout	10 Sekunden je Versuch
Maximale Versuche	5
Wiederholungsabstände	30 s · 2 min · 10 min · 1 h · 6 h
Weiterleitungen	werden nicht gefolgt (Schutz gegen Rebinding)
Gespeicherter Antwort-Rumpf	die ersten 2.000 Zeichen
Automatische Abschaltung	nach 8 aufeinanderfolgenden Fehlschlägen

Ein automatisch abgeschalteter Endpunkt lässt sich in der Console wieder aktivieren.

Die Zustellhistorie eines Webhooks ist über `GET /webhooks/{id}/deliveries/` abrufbar (Versuche, Status, Antwortzeit, nächste Wiederholung). Jede Zustellung führt u. a. `event_type`, `attempts`, `max_attempts`, `response_status`, `success`, `completed_at`.

Test-Webhook

`POST /webhooks/{id}/test/` sendet **synchron** einen Test-Request an die hinterlegte URL und gibt das Ergebnis direkt zurück:

```
{ "valid": true, "data": { "success": true, "status_code": 200 }, "meta": { "duration_ms": 342 } }
```

Ist die URL nicht erreichbar, antwortet die API mit `success: false` und einer Fehlerbeschreibung (HTTP 502).

Sicherheitsanforderungen an die Ziel-URL

- Nur **HTTPS**.
- Nur **öffentlich erreichbare** Adressen (interne/lokale Hosts werden abgelehnt — SSRF-Schutz).
- Die Prüfung läuft nicht nur beim Anlegen, sondern **unmittelbar vor jedem Versand erneut** — damit greift ein nachträgliches Umbiegen des DNS-Namens auf eine interne Adresse (DNS-Rebinding) nicht.

Exporte

DATEV-Export — GET /api/v1/exports/datev/

Erzeugt die **Buchungsdaten** der finalisierten Rechnungen eines Zeitraums als CSV im DATEV-Format — direkt importierbar in die Finanzbuchhaltung bzw. durch den Steuerberater.

Parameter

Parameter	Pflicht	Format	Bemerkung
<code>date_from</code>	ja	YYYY-MM-DD	Beginn des Zeitraums
<code>date_to</code>	ja	YYYY-MM-DD	Ende des Zeitraums

Erforderlicher Scope: `exports:read`.

Ausgabe

Eigenschaft	Wert
Medientyp	<code>text/csv; charset=windows-1252</code>
Kodierung	Windows-1252 (CP1252) mit sicherer Transliteration — DATEV-kompatibel
Trennzeichen	Semikolon (;)
Text-Quoting	nur bei Bedarf (<code>QUOTE_MINIMAL</code>)
Dezimaltrennzeichen	Komma (deutsches Format, z. B. <code>595,00</code>)
Belegdatum	DDMM (z. B. <code>1806</code>)
Dateiname	<code>datev_{date_from}_{date_to}.csv</code>
Format	DATEV-Buchungsstapel, 118 Spalten
Umfang	alle finalisierten Rechnungen im Zeitraum (Entwürfe ausgeschlossen), kontogebunden je Konto

Die Datei beginnt mit einer Kopfzeile aus exakt 118 Spaltennamen, gefolgt von einer Buchungszeile je Rechnung. Die ersten Spalten:

#	Spaltenname
1	Umsatz (ohne Soll/Haben-Kz)
2	Soll/Haben-Kennzeichen
3	WKZ Umsatz
4	Kurs
5	Basis-Umsatz
6	WKZ Basis-Umsatz
7	Konto
8	Gegenkonto (ohne BU-Schlüssel)
9	BU-Schlüssel
10	Belegdatum

Weitere Spalten umfassen u. a. Belegfeld 1/2, Buchungstext, Beleginfo 1–8 und die KOST-Felder. Bei gemischten Steuersätzen erfolgt eine Aufteilung je Satz.

Debitorennummer. Das Feld *Gegenkonto* nutzt die am Kunden hinterlegte Debitorennummer. Fehlt sie (Altbestand), greift ein synthetischer Fallback (10000 + Kunden-ID), damit bestehende Exporte unverändert bleiben.

Zur Versionsangabe. Der Export schreibt die 118-spaltige Buchungsstapel-Feldreihe samt Spalten-Kopfzeile. Eine separate **EXTF-Metadatenzeile** (Formatname, Versionsnummer, Berater-/Mandanten-Nr., WJ-Beginn, SKR) wird **nicht** ausgegeben — es steckt also kein maschinenlesbarer Versionsmarker in der Datei. Eine Versionsangabe bezieht sich auf das Feld-Layout, nicht auf eine im Export hinterlegte Formatversion.

Fehler

HTTP	Bedingung	Code
400	date_from/date_to fehlt	schema
400	Datum nicht im Format YYYY-MM-DD	schema

Weitere Exporte

- **Rechnungsdokumente:** ZUGFeRD-PDF und XRechnung-/UBL-XML lassen sich pro Rechnung über die Rechnungs-Endpunkte herunterladen (siehe Endpoints).
- **Konto-Datenexport:** vollständiger Export der eigenen Kontodaten auf Anforderung (siehe Compliance & Archivierung).